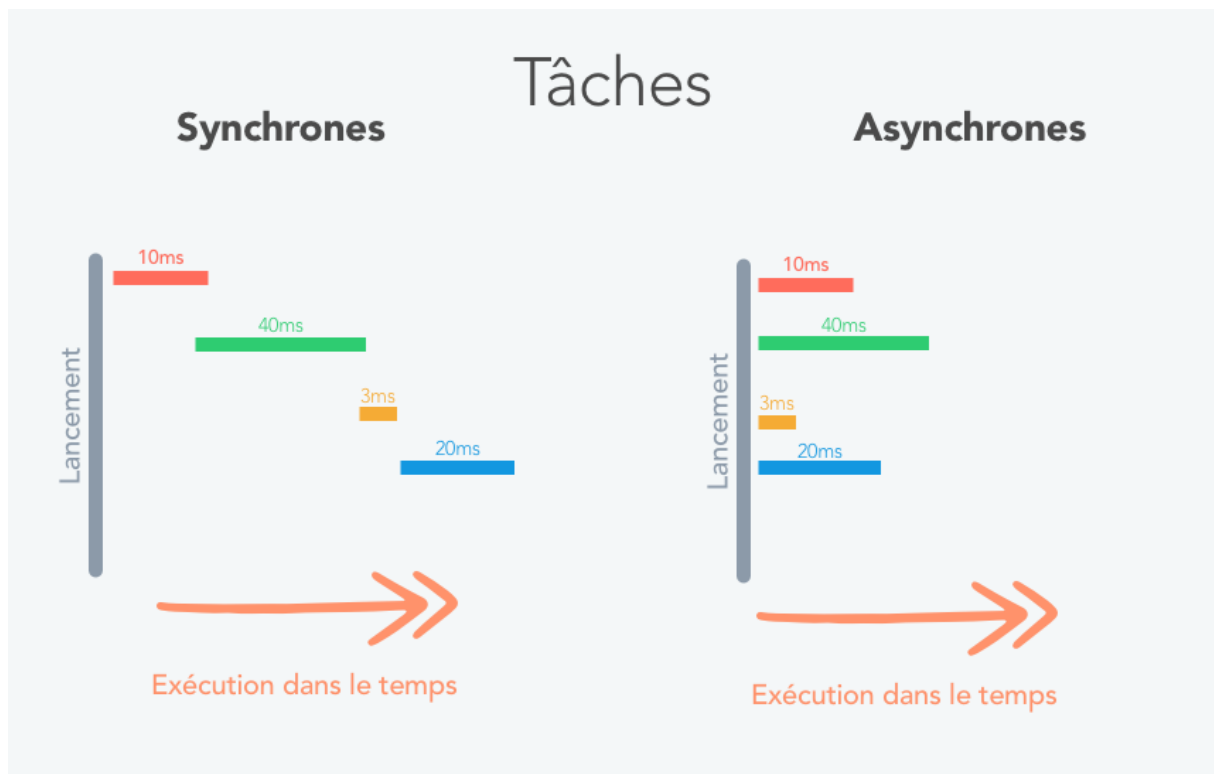


Thread JAVA

1. Introduction

Un thread est une portion de code capable de s'exécuter en parallèle à d'autres traitements (en parallèle sur une machine multiprocesseurs mais en temps partagé sur une machine monoprocesseur). Un thread n'est pas un processus. En effet, les processus vivent dans des espaces virtuels isolés alors que les threads sont des traitements qui vivent ensemble au sein d'un même processus : ils partagent la même mémoire contrairement aux processus. Les threads peuvent être utilisés par exemple pour :

- Faire des traitements en tâche de fond, c'est le cas de la coloration syntaxique des éditeurs.
- Exécuter plusieurs instances d'un même code pour accélérer le traitement, pour de longs traitements n'utilisant pas les mêmes ressources.
- S'adresser de manière personnelle à plusieurs clients simultanément comme les serveurs HTTP ou les Chat.
- Résoudre certaines problématiques liées au mode de fonctionnement de Swing. Sur les machines monoprocesseurs, les threads ne s'exécutent pas en même temps mais en temps partagé, c'est pour cette raison qu'il est important pour un Thread de toujours laisser une chance aux autres de s'exécuter. En Java, un thread est un objet d'une classe qui dispose d'une méthode nommée `run` qui sera exécutée lorsque le thread sera démarré. Deux façons de faire sont possibles : - une classe qui hérite de Thread - Une classe qui implémente l'interface Runnable



2. Implémentation des Thread :

En Java, un thread est un objet d'une classe qui dispose d'une méthode nommée run qui sera exécutée lorsque le thread sera démarré. Deux façons de faire sont possibles : - une classe qui hérite de Thread - Une classe qui implémente l'interface Runnable

1. Methode1

```
class PrimeThread extends Thread {
    long minPrime;
    PrimeThread(long minPrime) {
        this.minPrime = minPrime;
    }

    public void run() {
        // compute primes larger than minPrime
        . . .
    }
}
```

The following code would then create a thread and start it running:

```
PrimeThread p = new PrimeThread(143);
p.start();
```

2. Methode 2 :

```
class PrimeRun implements Runnable {
    long minPrime;
    PrimeRun(long minPrime) {
        this.minPrime = minPrime;
    }

    public void run() {
        // compute primes larger than minPrime
        . . .
    }
}
```

The following code would then create a thread and start it running:

```
PrimeRun p = new PrimeRun(143);
new Thread(p).start();
```

3. Methodes de la class Thread

- `currentThread()` donne le thread actuellement en cours d'exécution
 - `setName()` fixe le nom du thread
 - `getName()` nom du thread
 - `isAlive()` indique si le thread est actif(true) ou non (false)
 - `start()` lance l'exécution d'un thread
 - `run()` méthode exécutée automatiquement après que la méthode `start` précédente ait été exécutée
 - `stop()` : permet de terminer un thread , tuer le thread.
➔ `start()` et `stop()` ne peuvent être appelé qu'une fois !
 - `sleep(n)` :arrête l'exécution d'un thread pendant n millisecondes
 - `join()` opération bloquante - attend la fin du thread pour passer à l'instruction suivante
 - `interrupt()` : interrompt l'exécution d'un thread par un autre thread
 - `setPriority(int priorite)` : changer la priorité d'exécution d'un thread.
 - `wait()` : fait attendre le thread actuel jusqu'à ce qu'un autre thread invoque la méthode `notify()` ou la méthode `notifyAll()` pour cet objet
 - `notify()` : réveille un seul thread qui attend sur le moniteur de cet objet
-

Problème avec les threads :

Contrairement aux processus, les threads appartiennent au même programme. Ils peuvent donc partager des objets. Î deux threads qui accèdent au même objet peuvent créer un problème d'intégrité et de logique. Solution : Il faut "synchroniser" les threads

4. Les messages de synchronisation entre threads

La classe `Object` contient les méthodes `wait()`, `notify()` et `notifyAll()` pour permettre de synchroniser des threads grâce à l'envoi de messages. Ces méthodes permettent la mise en œuvre d'un mécanisme de communication par échanges de messages visant à synchroniser l'exécution de threads.

La méthode `wait()` met le thread courant en attente jusqu'à ce que l'objet reçoive une notification par les méthodes `notify()` ou `notifyAll()` : cette attente peut donc être potentiellement infinie.

La méthode `wait()` possède deux surcharges :

- `wait(long timeout)` : attend au plus la durée en millisecondes fournie en paramètre
- `wait(long timeout, int nanos)` : attend au plus la durée en millisecondes cumulée avec celle en nanosecondes fournies en paramètres

La méthode `notifyAll()` avertit tous les threads dont les méthodes `wait()` de la même instance sont invoquées.

La méthode `notify()` avertit un des threads dont la méthode `wait()` de la même instance est invoquée.

5. Synchronisation des threads

1. Méthodes synchronisées

Pour que les threads n'accèdent pas au même objet en même temps, on peut utiliser des méthodes mutuellement exclusives qu'on appelle en java méthodes synchronisées.

Avec le mot clé synchronized, à un instant donné, une seule méthode synchronisée peut donc accéder à un objet donné O : le verrou sur O est attribué à la méthode synchronisée appelée par l'objet O et il est restitué à la sortie de la méthode (à la sortie de la méthode l'objet O n'est plus verrouillé). Tant que le verrou n'est pas restitué pour cet objet, aucune autre méthode synchronisée ne peut recevoir le verrou sur cet objet concerné.

Remarque : les méthodes non synchronisées peuvent, quant à elles, accéder à tout moment à l'objet

2. Instruction synchronized

L'instruction synchronized permet d'acquérir un verrou sur un objet quelconque (qu'on cite dans l'instruction) pour une durée limitée à l'exécution d'un simple bloc

Syntaxe : synchronized(objet) {
 // Instructions de manipulation sur l'objet }